



Cross-GPU Reproducibility

Đàm Nguyên Khánh
Latex: Nguyễn Thế Hào

I. Giới thiệu

Trong nghiên cứu và triển khai mô hình học sâu, **reproducibility** gần như là một yêu cầu mặc định: ta cần chạy lại để kiểm chứng một ý tưởng, cần đồng đội reproduce kết quả, hoặc cần đảm bảo mô hình trong production không thay đổi hành vi “khó giải thích” sau mỗi lần retrain. Một kỳ vọng rất phổ biến là: chỉ cần **set random seed** thì kết quả sẽ tái lập. Nhưng trong thực tế, khi chuyển cùng một pipeline training sang **phần cứng khác** (GPU/TPU khác), kết quả đôi khi vẫn lệch dù code, dữ liệu và hyperparameters đều giữ nguyên.

🎯 Ví dụ thực tế

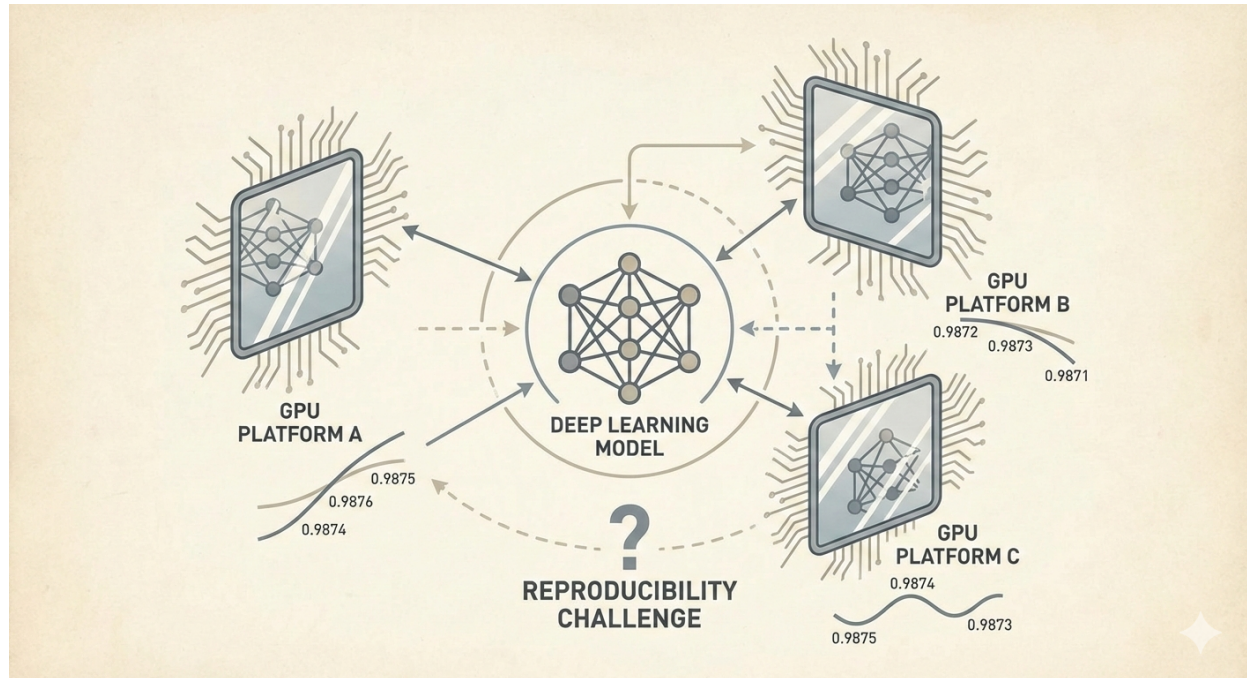
Ví dụ, bạn đặt **random seed = 42**, train trên GPU của mình và đạt test accuracy **77.92%**. Bạn gửi code cho đồng nghiệp, họ chạy trên một GPU khác với cùng seed, nhưng kết quả lại là **78.45%** — chênh lệch **0.53%**.

Vậy câu hỏi trung tâm của bài viết này là: **sự khác biệt này đến từ đâu và kiểm soát bằng cách nào?**

📖 Định nghĩa: Cross-GPU reproducibility

Trong bài viết này, **cross-GPU reproducibility** là mức độ “tái lập” khi ta **giữ nguyên mô hình và thiết lập training** nhưng **thay đổi phần cứng** (GPU/TPU).

- **Bit-exact reproducibility:** kết quả giống nhau *bit-by-bit*; thường rất khó đạt khi đổi GPU/TPU.
- **Statistical reproducibility:** mô hình học theo *cùng xu hướng* và đạt performance tương đương; đây là mục tiêu thực tế hơn trong đa số trường hợp.
- **Cách đánh giá thực tế:** không chỉ nhìn accuracy cuối, mà xem thêm training trajectory (ví dụ correlation giữa các đường cong theo epoch).



Hình 1: Minh họa thách thức reproducibility

Quan sát Hình 1, ta thấy một “độ lệch nhỏ” là hoàn toàn có thể xảy ra dù mô hình giống hệt nhau. Trong nhiều trường hợp, độ lệch này đến từ bản chất của *floating-point arithmetic*, thứ tự thực thi song song, và các tối ưu mặc định theo phần cứng.

Bài viết này là kết quả dựa trên **65 thực nghiệm** trên **6 GPU/TPU** (T4, L4, A100, RTX 3060, RTX 5090, TPU v6e-1), chúng ta sẽ cùng phân tích nguyên nhân gốc rễ và đưa ra checklist cấu hình thực tế để đạt reproducibility tốt nhất có thể.

II. Thực nghiệm: Số liệu và thiết lập

Để mô tả rõ vấn đề cross-GPU reproducibility, ta thực hiện **65 thực nghiệm** trên **6 GPU/TPU** khác nhau: Tesla T4, NVIDIA L4, NVIDIA A100-SXM4-40GB, NVIDIA GeForce RTX 3060, NVIDIA GeForce RTX 5090, và TPU v6e-1. Mỗi thực nghiệm sử dụng cùng mô hình **SimpleCNN** trên dataset **CIFAR-10**, cùng random seed (**42**) và cùng hyperparameters.

II.1. Mô tả vấn đề qua số liệu thực nghiệm

⇌ Cấu hình baseline dùng để so sánh

Để giảm các nguồn non-determinism phổ biến, ta dùng **baseline config**:

- TF32=False
- AMP=False
- num_workers=0

Với cùng baseline config, test accuracy cuối cùng vẫn khác nhau giữa các GPU/TPU.

Bảng 1: Kết quả baseline theo GPU/TPU (test accuracy cuối cùng)

GPU/TPU	Mean Accuracy	Std	Min	Max
TPU	78.45%	0.005%	78.45%	78.46%
NVIDIA GeForce RTX 3060	77.92%	0.24%	77.63%	78.28%
NVIDIA L4	77.85%	0.10%	77.75%	77.95%
NVIDIA GeForce RTX 5090	77.81%	0.07%	77.74%	77.91%
NVIDIA A100-SXM4-40GB	77.71%	0.10%	77.61%	77.81%
Tesla T4	77.64%	0.05%	77.59%	77.68%

Từ Bảng 1, ta có thể thấy **max difference** là **0.87%** (giữa TPU 78.45% và T4 77.64%). Điều này cho thấy: **cùng seed và cùng cấu hình vẫn chưa đủ để đảm bảo kết quả trùng khít giữa các phần cứng khác nhau.**

Lưu ý

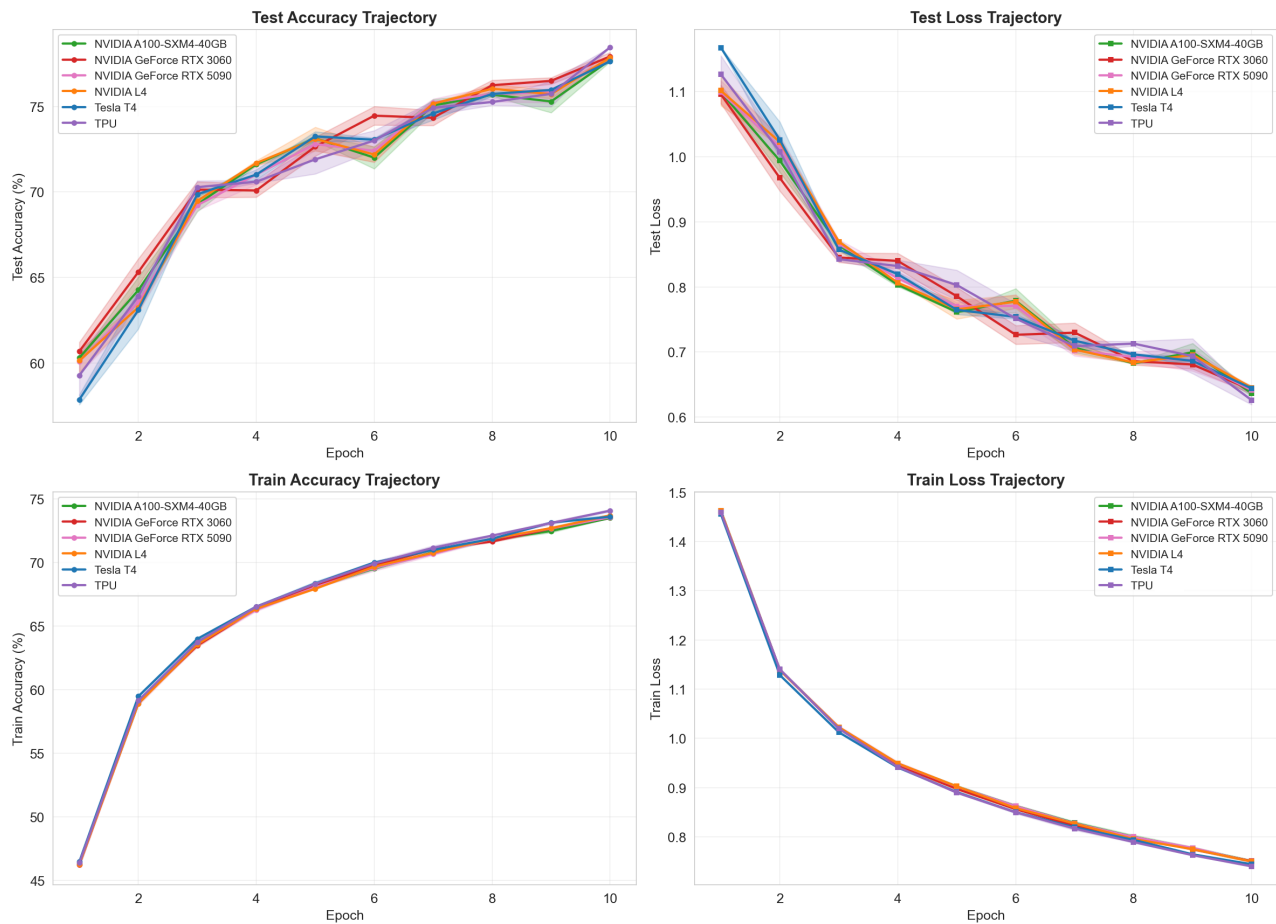
Lưu ý về Std của TPU: Std của TPU rất nhỏ (0.005%) vì chỉ có 2 thực nghiệm với baseline config (TF32=False, AMP=False, num_workers=0). Hai thực nghiệm này dùng precision khác nhau:

- FP32 (bf16_enabled=False): 78.46%
- BFloat16 (bf16_enabled=True): 78.45%

TPU cho thấy dấu hiệu reproducibility cao trong phạm vi thực nghiệm này (lệch 0.01% khi đổi FP32 $\leftrightarrow$ BF16), tuy nhiên số lượng runs còn hạn chế nên chưa thể kết luận tổng quát.

Ngoài accuracy cuối cùng, ta cũng quan sát **training trajectory** (accuracy theo epoch). Kết quả cho thấy xu hướng học tập giữa các GPU/TPU là rất tương đồng: correlation giữa các trajectories đạt **0.978–0.999**. Nói cách khác, mô hình có xu hướng “học giống nhau”, nhưng bị lệch nhẹ về giá trị tuyệt đối do khác biệt số học và cách thực thi trên phần cứng.

Training Trajectory Comparison (Cùng config: TF32=False, AMP=False, num_workers=0)



Hình 2: Training curves so sánh giữa các GPU/TPU

Quan sát Hình 2, ta thấy các đường cong accuracy có hình dạng tương tự nhau: tăng nhanh ở giai đoạn đầu, đạt peak khoảng epoch 7–8, rồi ổn định. Điều này gợi ý rằng vấn đề nằm ở **cross-hardware numerical differences**, không phải do mô hình học theo một “quỹ đạo” hoàn toàn khác.

II.2. Phương pháp và thiết lập thực nghiệm

Mục tiêu của thiết kế thực nghiệm là: (i) xác nhận vấn đề cross-GPU reproducibility thực sự tồn tại, (ii) cô lập các nguồn non-determinism phổ biến, và (iii) đo mức độ nhạy của các thành phần (cấu hình, precision, optimizer) đối với sai số số học.

Các thực nghiệm được chạy trên hai môi trường chính nhằm đảm bảo tính đa dạng phần cứng và khả năng tái lập.

- **Google Colab:** dùng GPU T4, L4, A100 và TPU v6e-1 cho baseline và một phần test cấu hình.
- **Phòng lab:** dùng RTX 3060 và RTX 5090 cho các test chi tiết về cấu hình và optimizer sensitivity.



Ba nhóm thực nghiệm chính

- **Baseline experiments (16 thực nghiệm):** đo mức chênh lệch “tự nhiên” giữa các GPU/TPU khi giữ cấu hình baseline.
- **Deterministic config tests (31 thực nghiệm):** thay đổi từng yếu tố (TF32, AMP, num_workers, cuDNN benchmark, deterministic algorithms) để cô lập nguyên nhân gây lệch.
- **Optimizer sensitivity tests (18 thực nghiệm):** so sánh Adam, RMSProp, SGD để đánh giá optimizer nào khuếch đại sai số số học mạnh hơn.

Thiết lập được giữ nhất quán để đảm bảo tính công bằng trong so sánh:

- **Model:** SimpleCNN (có BatchNorm và Dropout).
- **Dataset:** CIFAR-10 (50,000 training, 10,000 test).
- **GPUs/TPUs:** T4, L4, A100, RTX 3060, RTX 5090, TPU v6e-1.
- **Cấu hình test:** TF32 (on/off), AMP (on/off), num_workers (0/4), cuDNN benchmark (on/off), deterministic algorithms (on/off).
- **Optimizers:** Adam, RMSProp, SGD.
- **Metrics:** train/test loss, accuracy theo epoch (lưu lại đầy đủ để phân tích trajectory và correlation).

Vì model, dữ liệu, seed và hyperparameters được giữ cố định, mọi khác biệt quan sát được có thể quy về **khác biệt phần cứng** và/hoặc **khác biệt cấu hình thực thi**. Phần tiếp theo sẽ lần lượt kiểm tra các giả thuyết để truy ra “thủ phạm” chính.

III. Điều tra nguyên nhân

Sau khi xác nhận rằng cùng mô hình và cùng seed vẫn có thể cho kết quả khác nhau giữa các GPU/TPU, ta tiếp tục kiểm tra các giả thuyết phổ biến nhất gây ra non-determinism và chênh lệch số học. Mục tiêu của phần này là xác định “thủ phạm” chính và rút ra cấu hình thực tế giúp reproducibility tốt hơn.

III.1. Cấu hình mặc định khác nhau (TF32)

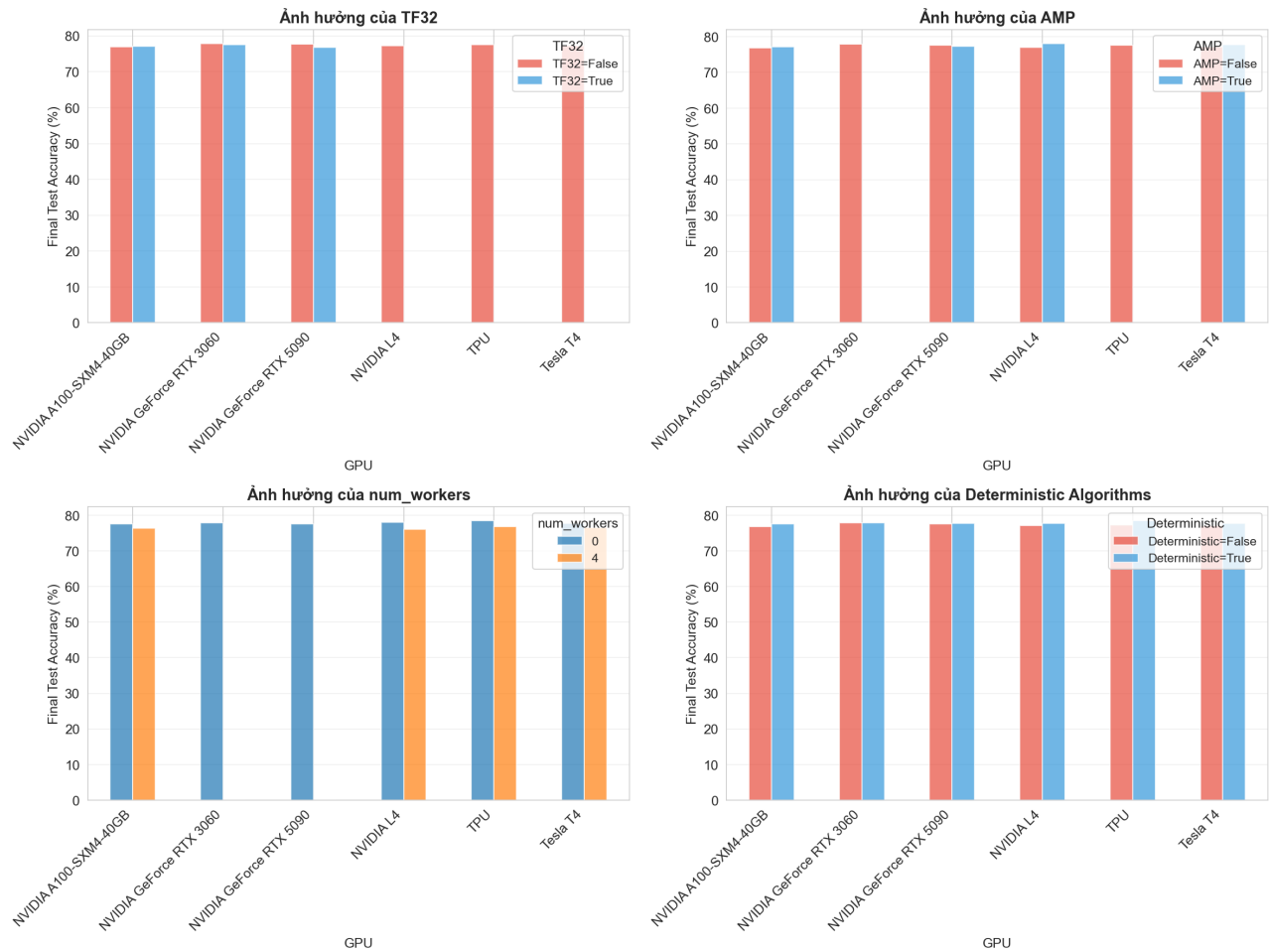
TF32 (TensorFloat-32)

TF32 là định dạng 19-bit do NVIDIA phát triển cho Tensor Cores. TF32 dùng cùng exponent với FP32 (8 bit) nhưng chỉ có 10 bit mantissa, vì vậy có thể tăng tốc đáng kể cho các phép toán ma trận với độ chính xác gần FP32. Trên nhiều GPU Ampere+ (ví dụ A100, RTX 30xx/40xx/50xx), TF32 có thể được bật mặc định trong một số phép toán.

Giả thuyết đặt ra là: nếu TF32 được bật mặc định trên một số GPU nhưng không tồn tại trên GPU khác, thì cùng một pipeline training có thể sử dụng precision hiệu dụng khác nhau, dẫn đến chênh lệch kết quả.

Kết quả quan sát cho thấy TF32 ảnh hưởng khác nhau theo từng kiến trúc:

- **GPU Ampere+ (A100, RTX 3060, RTX 5090):** TF32 có thể mặc định bật → khi tắt TF32, accuracy thay đổi nhẹ.
- **GPU Turing (T4):** không hỗ trợ TF32 → không có tùy chọn TF32.
- **GPU Ada (L4):** hỗ trợ TF32 nhưng mức ảnh hưởng có thể khác Ampere+ do khác kiến trúc/implementation.
- **TPU:** không có TF32 (tính năng riêng của NVIDIA) → thường dùng BF16/FP32.



Hình 3: Ảnh hưởng của các cấu hình (TF32, AMP, num_workers, deterministic algorithms) đến accuracy.

Quan sát Hình 3, ta thấy TF32 là một nguồn khác biệt quan trọng giữa các GPU: khi một số GPU dùng TF32 trong các phép toán trọng yếu, kết quả cuối có thể lệch nhẹ so với khi ép về FP32 thuần.

💡 Kết luận giả thuyết 1

TF32 (thường bật mặc định trên một số GPU Ampere+) → khác biệt precision hiệu dụng so với GPU không hỗ trợ TF32. Vì vậy, **tắt TF32** là một bước quan trọng khi cần so sánh công bằng và tăng reproducibility.

III.2. DataLoader non-deterministic (num_workers)

DataLoader và num_workers

num_workers quyết định số worker process dùng để load data:

- num_workers = 0: load data bằng main process → thứ tự và hành vi thường ổn định hơn.
- num_workers > 0: load data đa tiến trình → có thể gây non-determinism do scheduling và timing.

Trong bài viết này, baseline config đã đặt num_workers=0 để giảm non-determinism. Khi tăng num_workers, thứ tự batch/augmentation có thể bị ảnh hưởng bởi timing giữa các worker, từ đó làm lệch gradient và tích lũy chênh lệch qua epochs.

Kết luận giả thuyết 2

num_workers>0 có thể làm reproducibility kém hơn. Nếu mục tiêu ưu tiên reproducibility, giữ num_workers=0 là lựa chọn an toàn và dễ kiểm soát.

III.3. CuDNN benchmark chọn kernel khác nhau

cuDNN Benchmark

cuDNN benchmark là cơ chế tự động benchmark nhiều kernel và chọn kernel nhanh nhất. Khi benchmark=True, kernel được chọn có thể thay đổi do timing variations, từ đó gây non-determinism. Khi benchmark=False, cuDNN dùng kernel cố định hơn → thường deterministic hơn, đổi lại có thể giảm hiệu năng.

Quan sát thực nghiệm cho thấy bật benchmark=True có thể làm kết quả dao động nhiều hơn giữa các runs, đặc biệt khi kết hợp với các yếu tố khác như AMP và multi-worker data loading.

Kết luận giả thuyết 3 (trade-off)

Tắt cuDNN benchmark giúp tăng deterministic nhưng có thể làm giảm performance. Nếu bạn cần reproducibility cao (benchmark, paper submission), nên ưu tiên benchmark=False.

III.4. AMP (Automatic Mixed Precision)

AMP (Automatic Mixed Precision)

AMP tự động dùng mixed precision để tăng tốc: thường dùng FP16/BF16 cho một phần forward pass và giữ FP32 cho các phần nhạy cảm (ví dụ accumulation/backward). AMP giúp giảm bộ nhớ và tăng tốc training, nhưng có thể tạo ra sai khác số học nhỏ so với FP32 thuần.

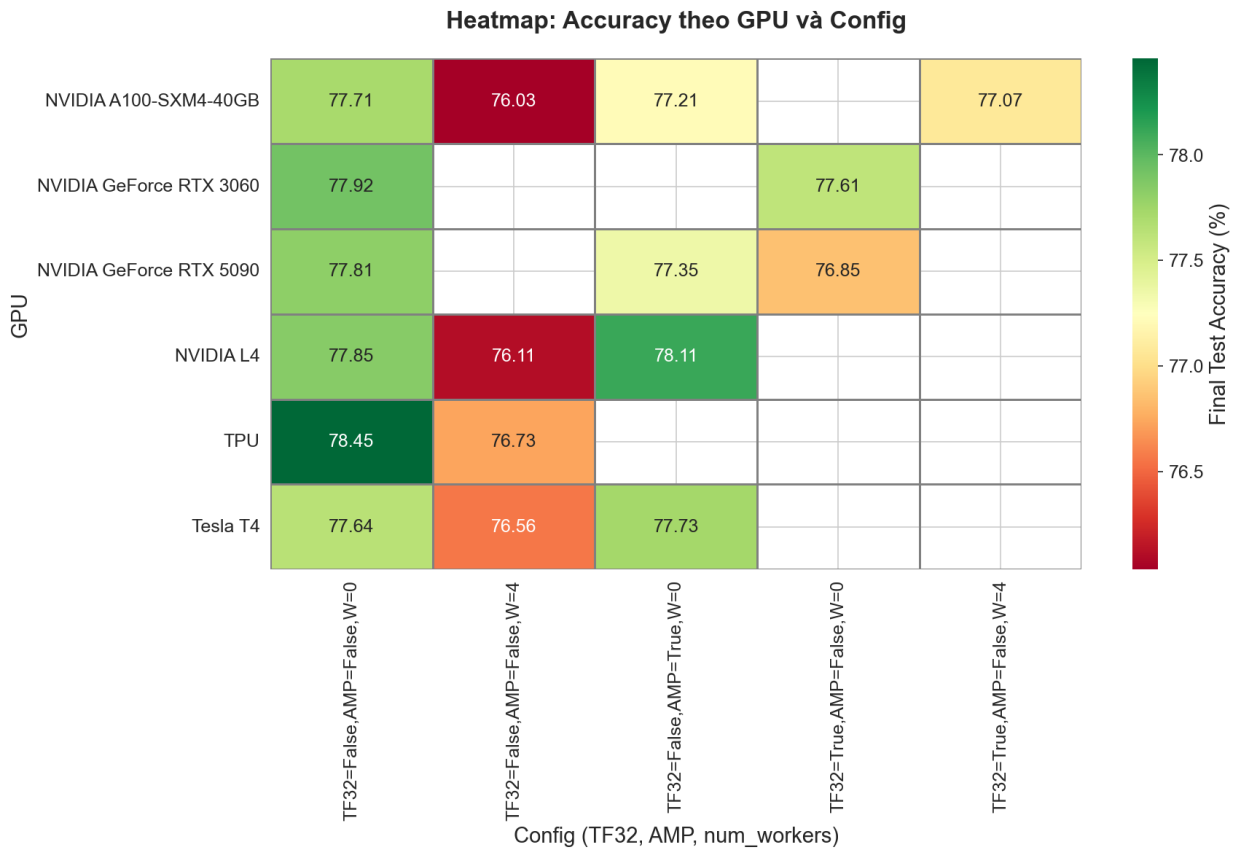
Trong thực nghiệm, AMP có thể làm accuracy thay đổi và làm tăng độ nhạy của training, tuy nhiên xu hướng học tập giữa các GPU vẫn tương đối tương đồng nếu các cấu hình còn lại được kiểm soát tốt.

💡 Kết luận giả thuyết 4

AMP có thể ảnh hưởng đến reproducibility do thay đổi precision hiệu dụng. Nếu mục tiêu là reproducibility cao, nên **tắt AMP** trong baseline để so sánh công bằng. Nếu mục tiêu là performance, có thể bật AMP nhưng cần chấp nhận trade-off về reproducibility.

III.5. Tổng hợp: Heatmap ảnh hưởng theo GPU và cấu hình

Để có cái nhìn tổng quan, ta tổng hợp accuracy theo GPU và theo từng cấu hình dưới dạng heatmap. Cách nhìn này giúp nhận ra cấu hình nào làm kết quả ổn định hơn, và cấu hình nào dễ tạo chênh lệch.



Hình 4: Heatmap accuracy theo GPU và cấu hình.

Quan sát Hình 4, ta thấy cấu hình baseline (TF32=False, AMP=False, num_workers=0) thường cho kết quả ổn định nhất trong phạm vi thực nghiệm. Từ đây, ta có thể ưu tiên baseline làm điểm chuẩn, sau đó mới bật dần các tối ưu (AMP, benchmark, multi-workers) tùy vào mục tiêu performance.

 Key insight từ phần điều tra

Khác biệt precision (đặc biệt TF32) và các cơ chế chọn kernel (cuDNN benchmark) là hai nguồn gây khác biệt quan trọng. Để tăng reproducibility, ưu tiên: TF32=False, AMP=False, num_workers=0, và cân nhắc benchmark=False khi cần deterministic cao.

IV. Hiểu sâu vấn đề

Ở phần trước, ta đã thấy rằng ngay cả khi cố gắng “đồng bộ” cấu hình giữa các GPU, kết quả cuối vẫn có thể lệch nhẹ. Phần này đi sâu vào nguyên nhân cốt lõi: **sai khác số học (floating-point)** và cách mà **optimizer** có thể khuếch đại sai khác nhỏ đó qua nhiều epochs.

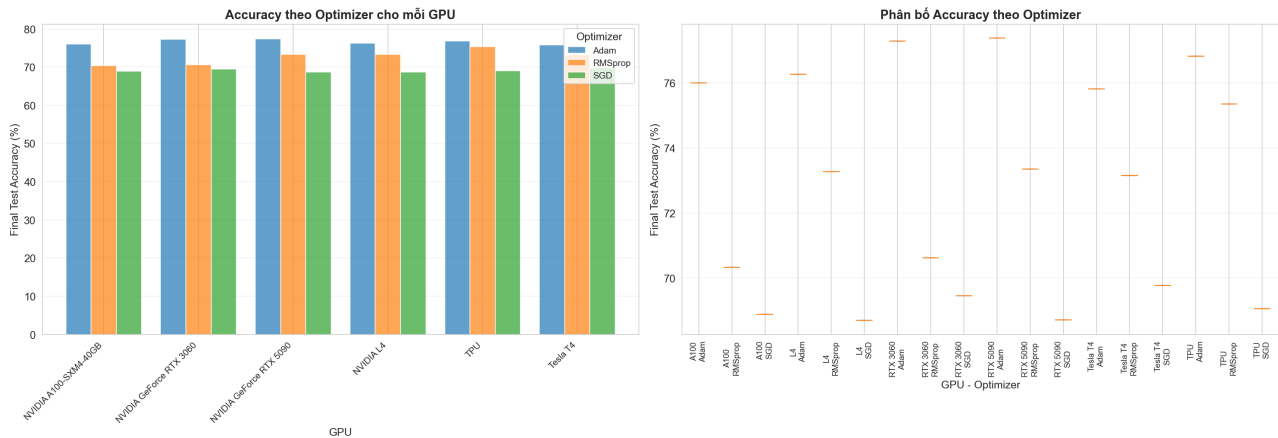
IV.1. Optimizer sensitivity: Sai số nhỏ có thể bị khuếch đại

Một phát hiện đáng chú ý là các optimizer khác nhau có độ nhạy khác nhau với sai khác số học. Trong thực nghiệm, ta so sánh ba optimizer phổ biến: Adam, RMSProp và SGD.

Bảng 2: Optimizer sensitivity (mean accuracy across all GPUs)

Optimizer	Mean Accuracy	Độ nhạy cảm
Adam	76.60%	Trung bình
RMSProp	72.68%	Cao
SGD	69.09%	Thấp

Ghi chú: So sánh trong Bảng 1 nhằm đánh giá độ nhạy với sai số số học, không nhằm khẳng định hiệu năng tối ưu của từng optimizer khi được tune đầy đủ.



Hình 5: So sánh accuracy theo optimizer cho mỗi GPU/TPU.

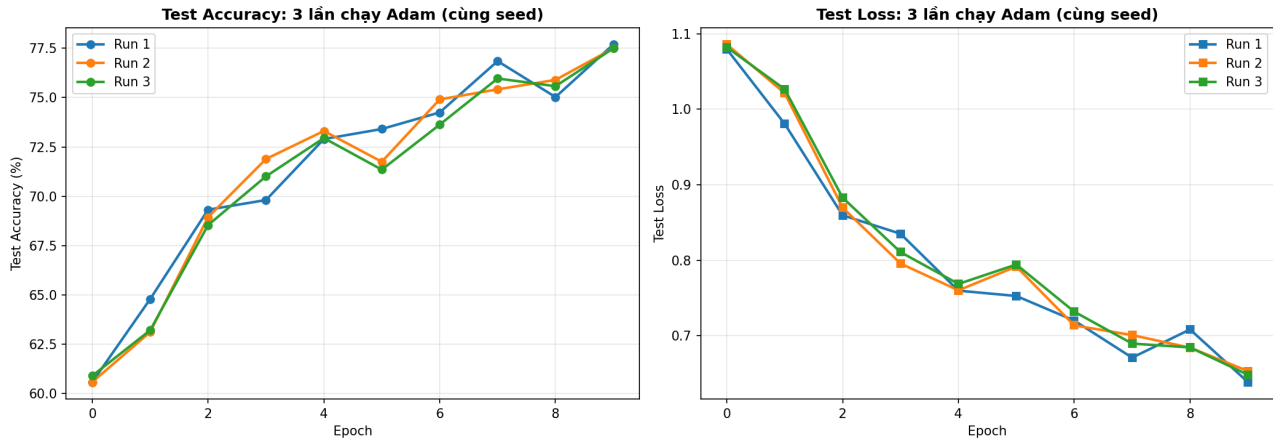
Quan sát Hình 5, ta thấy Adam cho accuracy tốt hơn trên hầu hết GPU/TPU, trong khi RMSProp và SGD thấp hơn đáng kể trong cấu hình thực nghiệm này. Tuy nhiên, điểm quan trọng của phần này không phải “optimizer nào tốt nhất”, mà là: **optimizer nào dễ khuếch đại sai khác số học hơn**.

Cơ chế tổng quát có thể tóm tắt như sau: các **adaptive optimizer** (như Adam, RMSProp) sử dụng momentum và adaptive learning rate, vì vậy một sai khác nhỏ trong gradient (do floating-point precision) có thể: (i) đi vào momentum buffer, (ii) ảnh hưởng đến bước cập nhật theo thời gian, và (iii) tích lũy qua nhiều epochs. Hiệu ứng này giống một “butterfly effect”: sai số nhỏ ban đầu có thể dẫn đến khác biệt lớn hơn về sau.

Cơ chế khuếch đại sai số (adaptive optimizer)

Khi có một sai số nhỏ trong gradient (do floating-point precision), adaptive optimizer có thể:

1. Lưu trữ sai số này trong momentum buffer.
2. Dùng các thống kê tích lũy để tính learning rate theo từng tham số.
3. Khiến bước cập nhật ở các epoch sau lệch nhẹ so với lần chạy khác.
4. Tích lũy sai khác theo thời gian và dẫn đến lệch ở kết quả cuối.



Hình 6: Training trajectory của Adam trên RTX 5090 — minh họa sự tích lũy sai khác theo epoch.

Quan sát Hình 6, ta có thể thấy các lần chạy có thể bám sát nhau ở giai đoạn đầu nhưng vẫn có khả năng “tách” dần theo thời gian do sai khác nhỏ bị tích lũy.

💡 Kết luận: Optimizer sensitivity

Sai khác floating-point ban đầu có thể rất nhỏ, nhưng adaptive optimizer có thể khuếch đại sai khác này qua thời gian. Vì vậy, khi đánh giá reproducibility, cần so sánh cả **quỹ đạo học** (trajectory), không chỉ accuracy cuối cùng.

IV.2. Floating-point precision: Nguồn gốc của sai khác

Để hiểu vì sao sai khác số học xảy ra, ta cần nhìn vào floating-point precision và cách phần cứng thực thi phép tính.

Các định dạng precision (FP32/FP16/BF16)

Các định dạng phổ biến trong deep learning:

- **FP32:** 32-bit (1 sign, 8 exponent, 23 mantissa), độ chính xác khoảng 7 chữ số thập phân.
- **FP16:** 16-bit (1 sign, 5 exponent, 10 mantissa), nhanh và tiết kiệm bộ nhớ nhưng dễ mất chính xác.
- **BF16:** 16-bit (1 sign, 8 exponent như FP32, 7 mantissa), phạm vi giống FP32 nhưng ít chính xác hơn.

Trong thực tế, các GPU/TPU khác nhau hỗ trợ và ưu tiên các precision khác nhau. Điều này tạo ra sự khác biệt về “precision hiệu dụng” ngay cả khi code nhìn có vẻ giống nhau. Hai điểm mấu chốt khiến sai khác dễ xuất hiện:

- **Thứ tự tính toán (order of operations):** tính toán song song $\rightarrow$ thứ tự cộng dồn có thể khác nhau $\rightarrow$ sai khác làm tròn (rounding) khác nhau.
- **Tích lũy theo thời gian:** sai lệch nhỏ ở mỗi bước cập nhật $\rightarrow$ tích lũy qua epochs $\rightarrow$ có thể thành lệch đáng kể ở cuối training.

Kết luận: Floating-point differences

Khác biệt floating-point không nhất thiết là bug; đó là hệ quả tự nhiên của số học xấp xỉ và thực thi song song. Vì vậy, **reproducibility thực tế nên được đánh giá theo thống kê và quỹ đạo học**, không kỳ vọng bit-exact giữa mọi phần cứng.

IV.3. Statistical vs Bit-exact reproducibility

Định nghĩa: Bit-exact vs Statistical

- **Bit-exact reproducibility:** kết quả giống nhau *bit-by-bit*. Thường rất khó đạt khi đổi GPU/TPU, driver, cuDNN hoặc thay đổi kernel.
- **Statistical reproducibility:** mô hình học theo *cùng xu hướng* và đạt performance tương đương. Đây là mục tiêu thực tế trong đa số trường hợp.

Trong phạm vi thực nghiệm của bài viết này, baseline config (TF32=False, AMP=False, num_workers=0) cho thấy mức độ reproducibility theo hướng **statistical** là khả thi: accuracy cuối có thể lệch nhẹ, nhưng correlation của trajectory vẫn rất cao.

💡 Key insight của phần này

Nếu correlation của trajectory cao (ví dụ > 0.95) và chênh lệch accuracy nằm trong ngưỡng chấp nhận, ta có thể xem đó là **statistical reproducibility**. Trong đa số dự án, đây là mục tiêu hợp lý hơn so với bit-exact reproducibility.

V. Giải pháp thực tế

Sau khi xác định các nguồn gây chênh lệch (precision, DataLoader, cuDNN benchmark, AMP) và hiểu cơ chế tích lũy sai khác qua optimizer, ta có thể rút ra một checklist cấu hình giúp tăng reproducibility trong thực tế. Mục tiêu của phần này là đưa ra các bước **dễ áp dụng, dễ kiểm soát**, và có thể dùng như một “baseline” khi bạn cần so sánh công bằng giữa các phần cứng.

V.1. Cấu hình deterministic: Checklist thực tế

✓ Checklist deterministic (khuyến nghị dùng làm baseline)

Để reproducibility tốt nhất có thể khi train trên GPU, ta ưu tiên: tắt TF32, tắt cuDNN benchmark, bật deterministic algorithms, và dùng `num_workers=0`.

Code example (PyTorch): Deterministic Checklist

```

1 import torch
2 import numpy as np
3 from torch.utils.data import DataLoader
4
5 # 1. Set seed cho tất cả RNG
6 def set_seed(seed=42):
7     np.random.seed(seed)
8     torch.manual_seed(seed)
9     torch.cuda.manual_seed(seed)
10    torch.cuda.manual_seed_all(seed)
11
12 # 2. Tắt TF32 (đảm bảo FP32 nhất quán)
13 torch.backends.cudnn.allow_tf32 = False
14 if hasattr(torch.backends, "cublas"):
15     torch.backends.cublas.allow_tf32 = False
16
17 # 3. Tắt cuDNN benchmark (tránh thay đổi kernel theo timing)
18 torch.backends.cudnn.benchmark = False
19
20 # 4. Bật deterministic algorithms
21 torch.backends.cudnn.deterministic = True
22 torch.use_deterministic_algorithms(True, warn_only=True)
23
24 # 5. DataLoader: num_workers = 0 (ổn định thứ tự load)
25 train_loader = DataLoader(

```

```

26     dataset ,
27     batch_size=128 ,
28     shuffle=True ,
29     num_workers=0
30 )

```

Checklist trên tập trung vào các điểm dễ gây lệch nhất trong thực nghiệm: precision hiệu dụng (TF32), kernel selection (cuDNN benchmark), và non-deterministic data loading (`num_workers`). Trong nhiều trường hợp, chỉ cần áp dụng đầy đủ các bước này là đã đủ để xây dựng một baseline ổn định phục vụ so sánh.

💡 Kết luận nhanh

Nếu bạn cần reproducibility cao (benchmark, paper submission, so sánh mô hình), hãy ưu tiên dùng checklist deterministic làm baseline trước khi bật các tối ưu hiệu năng.

V.2. Chuẩn hoá môi trường: Những thứ dễ bị quên

Ngay cả khi đã bật deterministic config, kết quả vẫn có thể lệch nếu môi trường training không đồng nhất. Vì vậy, ta cần chuẩn hoá các yếu tố “phi-mã nguồn” sau đây.

⚙️ Checklist chuẩn hoá môi trường

- **Cùng batch size:** batch size khác nhau làm gradient khác nhau.
- **Cùng preprocessing:** normalization và augmentation phải giống nhau.
- **Cùng precision:** chọn một trong FP32/FP16/BF16 và giữ nguyên.
- **Cùng seed:** set seed cho numpy, torch và CUDA (bao gồm `manual_seed_all`).
- **Cùng phiên bản phần mềm:** PyTorch, CUDA, cuDNN, driver.

Trong thực tế, việc ghi rõ environment (GPU model, precision, versions) thường giúp tiết kiệm rất nhiều thời gian debug khi kết quả bị lệch giữa các máy.

V.3. Best practices cho nghiên cứu và báo cáo

💡 Best practices (tóm tắt)

- **So sánh xu hướng (trend), không chỉ so giá trị tuyệt đối:** xem training trajectory có tương đồng không; correlation cao là dấu hiệu quan trọng.
- **Ghi rõ GPU + precision + config trong báo cáo:** ví dụ “NVIDIA A100-SXM4-40GB, FP32, TF32=False, AMP=False”.
- **Không debug chỉ dựa trên accuracy:** luôn kiểm tra thêm loss curves và trajectory.
- **Dùng deterministic config khi cần reproducibility cao:** chấp nhận trade-off về performance để đổi lấy tính tái lập.

Một nguyên tắc thực tế là: nếu bạn đang báo cáo kết quả cho người khác (paper, blog, benchmark nội bộ), hãy mô tả đầy đủ phần cứng và config. Nếu bạn đang tối ưu performance cho một hệ thống riêng, bạn có thể bật dần các tối ưu nhưng vẫn nên giữ một baseline deterministic để đối chiếu.

VI. Kết luận

Qua các phần trước, ta đã thấy rằng **cross-GPU reproducibility** là một vấn đề có thật: ngay cả khi giữ nguyên mô hình, dữ liệu, hyperparameters và random seed, kết quả training vẫn có thể lệch khi thay đổi GPU/TPU. Điểm quan trọng là hiểu đúng bản chất của sự lệch này để chọn cách đánh giá và cấu hình phù hợp.

💡 Key takeaways

1. **Cross-GPU reproducibility thường là statistical, không phải bit-exact:** mục tiêu thực tế là đảm bảo mô hình học theo *cùng xu hướng* (trajectory tương đồng), thay vì yêu cầu kết quả giống nhau bit-by-bit giữa mọi phần cứng.
2. **So sánh xu hướng quan trọng hơn giá trị tuyệt đối:** hãy xem thêm loss/accuracy curves theo epoch và correlation giữa trajectories, thay vì chỉ nhìn accuracy cuối cùng.
3. **Khác biệt precision và tối ưu mặc định là nguồn lệch lớn:** TF32, AMP, cuDNN benchmark và thứ tự thực thi song song có thể tạo sai khác số học, rồi sai khác này tích lũy dần qua epochs.
4. **Optimizer có thể khuếch đại sai khác:** các adaptive optimizer (Adam, RMSProp) dễ tích lũy sai khác nhỏ qua momentum/adaptive update, nên cần kiểm soát config khi so sánh giữa các GPU.
5. **Xác định phần cứng và cấu hình ngay từ đầu:** ghi rõ GPU/TPU, precision, TF32/AMP, cuDNN benchmark, deterministic algorithms, và versions (PyTorch/CUDA/cuDNN) giúp giảm rất nhiều thời gian debug về sau.

💡 Hướng dẫn nhanh: Để reproducibility tốt nhất

- Tắt TF32: `torch.backends.cudnn.allow_tf32 = False`
- Tắt cuDNN benchmark: `torch.backends.cudnn.benchmark = False`
- `num_workers = 0` trong DataLoader
- Bật deterministic algorithms: `torch.use_deterministic_algorithms(True, warn_only=True)`

💡 Hướng dẫn nhanh: Để performance tốt nhất

- Bật TF32 (nếu GPU hỗ trợ)
- Bật cuDNN benchmark
- `num_workers > 0`
- Bật AMP (nếu phù hợp với mô hình và pipeline)

 Hướng dẫn nhanh: Cân bằng (khuyến nghị cho đa số trường hợp)

- Tắt TF32
- `num_workers = 0`
- Bật cuDNN benchmark nếu cần thêm performance
- Chỉ bật AMP khi bạn chấp nhận trade-off về reproducibility

Tóm lại, trong bối cảnh deep learning hiện đại, thay vì kỳ vọng bit-exact giữa mọi phần cứng, ta nên ưu tiên một tiêu chuẩn thực tế hơn: **statistical reproducibility**, tức mô hình học theo cùng learning dynamics và đạt performance tương đương. Điều này vừa phù hợp với bản chất floating-point/parallel computation, vừa giúp ta tập trung vào việc đánh giá đúng chất lượng mô hình trong thực tế.

VII. Code & Dữ liệu thực nghiệm

 Tài nguyên để reproduce kết quả

Toàn bộ code và notebook thực nghiệm trong bài viết này được công bố trong GitHub repository: **AIO2025_Blogs/251226 Blog Hardware 2**.

Repository bao gồm:

- Notebook training trên từng GPU/TPU.
- Code tổng hợp và phân tích kết quả.
- Các file JSON chứa kết quả thực nghiệm (loss/accuracy theo epoch, thống kê tổng hợp).

Nhờ đó, người đọc có thể **reproduce** và **verify** toàn bộ findings được trình bày trong bài viết.

VIII. Tài liệu tham khảo

- [1] PyTorch Team, *Pytorch documentation - reproducibility*, PyTorch Documentation, 2025. [Online]. Available: <https://pytorch.org/docs/stable/notes/randomness.html>.
- [2] NVIDIA Corporation, *Nvidia gpu documentation - tf32, mixed precision training*, NVIDIA Developer Documentation, 2025. [Online]. Available: <https://blogs.nvidia.com/blog/tenorfloat-32-precision-format/>.
- [3] PyTorch Team, *Pytorch documentation - tf32 on ampere and later devices*, PyTorch Documentation, 2025. [Online]. Available: <https://pytorch.org/docs/stable/notes/cuda.html#tf32-on-ampere>.
- [4] N. P. Jouppi et al., “In-datacenter performance analysis of a tensor processing unit”, in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2017. [Online]. Available: <https://arxiv.org/abs/1704.04760>.
- [5] PyTorch Team, *Pytorch documentation - cuda determinism*, PyTorch Documentation, 2025. [Online]. Available: <https://pytorch.org/docs/stable/notes/cuda.html#reproducibility>.
- [6] NVIDIA Corporation, *Cudnn documentation - deterministic algorithms*, NVIDIA Developer Documentation, 2025. [Online]. Available: <https://docs.nvidia.com/deeplearning/cudnn/>.